

Dunaújvárosi Egyetem Bánki Donát Technikum

Projekt feladat dokumentáció

Tartalom

Az ötlet rövid leírása:	1
Működési elv	1
Adatok rögzítése	2
Ügyfél regisztráció	2
Küldemény nyilvántartás	2
MySQL adatbázis	2
Fejlesztési lehetőségek	3
Web felület készítése	3
Mobil alkalmazás	3
Automatikus értesítések	4
Hozzáférési szintek kezelése	4
Rendszer tesztelése	4
Önreflexió	4

Tantárgy neve: Adatbáziskezelés

Projekt tervezői: Batári Péter

Projekt címe: Webshop

Osztály: 12.B

Dátum: 2025.05.23.

Az ötlet rövid leírása:

A projekt egy **webshop háttérrendszerének adatbázisa**, amely alkalmas felhasználók kezelésére, termékek nyilvántartására, rendelések rögzítésére és azok állapotának követésére.

A rendszer célja egy **strukturált, normalizált MySQL adatbázis** megvalósítása, amely biztonságosan és hatékonyan kezeli az üzleti adatokat.

Működési elv

A webshop működése során:

- a felhasználók regisztrálnak és bejelentkeznek,
- termékeket böngésznek,
- rendeléseket adnak le,

- az adminisztrátor kezeli a termékeket és a rendelések státuszát.

Adatok rögzítése

Az adatbázis az alábbi adatokat tárolja:

- felhasználói adatok (név, e-mail, jelszó hash)
- termékek adatai (név, ár, készlet, kategória)
- rendelések és rendelési tételek
- fizetési és szállítási információk

Ügyfél regisztráció

Az ügyfél regisztráció során:

- egyedi e-mail cím kerül rögzítésre
- jelszó titkosított formában tárolódik
- alapértelmezett felhasználói szerepkör kerül hozzárendelésre

Az adatbázis biztosítja, hogy egy e-mail cím csak egyszer szerepelhessen a rendszerben.

Küldemény nyilvántartás

A rendelésekhez kapcsolódóan:

- minden rendeléshez tartozik szállítási státusz
- nyomon követhető a rendelés állapota (pl. feldolgozás alatt, kiszállítva)
- a rendelések időbélyeggel vannak ellátva

Ez lehetővé teszi a rendelések visszakeresését és státusz szerinti lekérdezését.

MySQL adatbázis

```
CREATE DATABASE webshop;
USE webshop;

-- Szerepkörök
CREATE TABLE roles (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE
);

-- Felhasználók
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role_id INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (role_id) REFERENCES roles(id)
);

-- Kategóriák
CREATE TABLE categories (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE
);

-- Termékek (PÉLDA)
CREATE TABLE products (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL DEFAULT 'PÉLDA',
    price DECIMAL(10,2) NOT NULL CHECK (price >= 0),
    stock INT NOT NULL CHECK (stock >= 0),
    category_id INT,
    FOREIGN KEY (category_id) REFERENCES categories(id)
);

-- Rendelések
CREATE TABLE orders (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    status VARCHAR(50) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id)
);

-- Rendelési tételek
CREATE TABLE order_items (
    id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL CHECK (quantity > 0),
    price DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(id),
    FOREIGN KEY (product_id) REFERENCES products(id)
);

-- Szállítás / küldemény
```

```

FROM products
WHERE id = NEW.product_id;

IF current_stock < NEW.quantity THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'A PÉLDA termék nincs raktáron vagy nincs elegendő készlet.';
END IF;
END;
//

-- Készlet csökkentése sikeres rendelés után
CREATE TRIGGER decrease_stock_after_order
AFTER INSERT ON order_items
FOR EACH ROW
BEGIN
    UPDATE products
    SET stock = stock - NEW.quantity
    WHERE id = NEW.product_id;
END;
//

DELIMITER ;

-- Alap szerepkörök
INSERT INTO roles (name) VALUES ('user'), ('admin');

-- Szállítás / küldemény
CREATE TABLE shipments (
    id INT AUTO INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    shipping_status VARCHAR(50) NOT NULL,
    shipped_at TIMESTAMP NULL,
    FOREIGN KEY (order_id) REFERENCES orders(id)
);

-- Értesítések
CREATE TABLE notifications (
    id INT AUTO INCREMENT PRIMARY KEY,
    user_id INT,
    message VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    is_read BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (user_id) REFERENCES users(id)
);

-- Készletellenőrzés rendelés előtt
DELIMITER //

CREATE TRIGGER check_stock_before_order
BEFORE INSERT ON order_items
FOR EACH ROW
BEGIN
    DECLARE current_stock INT;

    SELECT stock INTO current_stock

```

Fejlesztési lehetőségek

Web felület készítése

A rendszer webes felülettel is rendelkezik, amely a felhasználók és az adminisztrátor számára biztosít hozzáférést a funkciókhoz:

- reszponzív kialakítás, amely asztali és mobil böngészőkön is használható
- felhasználói felület funkciói:
 - regisztráció és bejelentkezés
 - termékek listázása, kategóriák szerinti szűrés
 - termékadatok megtekintése
 - rendelés leadása és rendelési előzmények megtekintése
- adminisztrációs felület:
 - termékek és kategóriák kezelése (CRUD műveletek)
 - raktárkészlet módosítása és ellenőrzése
 - rendelések és szállítási státuszok kezelése
 - felhasználók jogosultságainak kezelése
- a webes felület közvetlen kapcsolatban áll az adatbázissal, biztonságos hitelesítési és jogosultsági mechanizmusokon keresztül
- a rendszer védi az adatokat SQL injection és jogosulatlan hozzáférés ellen

A webes felület biztosítja az átlátható kezelést, a gyors adatfeldolgozást és a felhasználóbarát működést.

Mobil alkalmazás

A rendszer kialakítása lehetővé teszi mobilalkalmazás csatlakoztatását:

- a backend adatbázis REST API-n keresztül elérhető
- a mobilalkalmazás képes:
 - felhasználói regisztrációra és bejelentkezésre
 - termékek böngészésére és keresésére
 - rendelések leadására és állapotuk követésére
- az adatbázis struktúrája platformfüggetlen, így webes és mobil kliens egyaránt használhatja

- a felhasználói munkamenetek és jogosultságok biztonságosan kezelhetők

Ez lehetővé teszi a rendszer későbbi bővítését Android és iOS alkalmazásokkal.

Automatikus értesítések

Az adatbázis támogatja riasztási és értesítési funkciók megvalósítását:

- alacsony készletszint esetén automatikus figyelmeztetés generálható
- rendelési státusz változásakor értesítés küldhető a felhasználónak
- adminisztrátori riasztások:
 - sikertelen fizetés
 - késedelmes szállítás
 - kritikus rendszerállapotok
- az értesítések időbélyeggel és státusszal kerülnek rögzítésre

A riasztások e-mailben, mobil push értesítésként vagy az admin felületen jelenhetnek meg.

Hozzáférési szintek kezelése

szerepkör alapú hozzáférés-vezérlés (RBAC) kerül alkalmazásra

alapértelmezett szerepkörök:

- **felhasználó** – termékek böngészése, rendelés leadása, saját adatok megtekintése
- **adminisztrátor** – termékek, rendelések, raktárkészlet és felhasználók kezelése

minden felhasználóhoz egy szerepkör tartozik, amely az adatbázisban kerül tárolásra
a jogosultságok ellenőrzése minden kritikus művelet előtt megtörténik

Rendszer tesztelése

CRUD műveleteken keresztül történik

idegen kulcsok és egyedi kulcsok működésének ellenőrzése

felhasználói regisztráció és bejelentkezés tesztelése

rendelési folyamat tesztelése

jogosultságok ellenőrzése

hibakezelés és határérték-tesztelés

Önreflexió

Az adatbázis tantárgy eleinte nehézkesen indult számomra, sokszor bizonytalan voltam a normalizálás és az SQL-lekérdezések miatt. A gyakorlati feladatok és a projektmunka során azonban egyre könnyebben értettem meg az összefüggéseket, és magabiztosabban tudtam kezelni az adatbázisokat. Most már látom, hogyan lehet egy rendszert strukturáltan felépíteni, és élvezem, hogy a tudásomat a saját projektemben is alkalmazhatom.